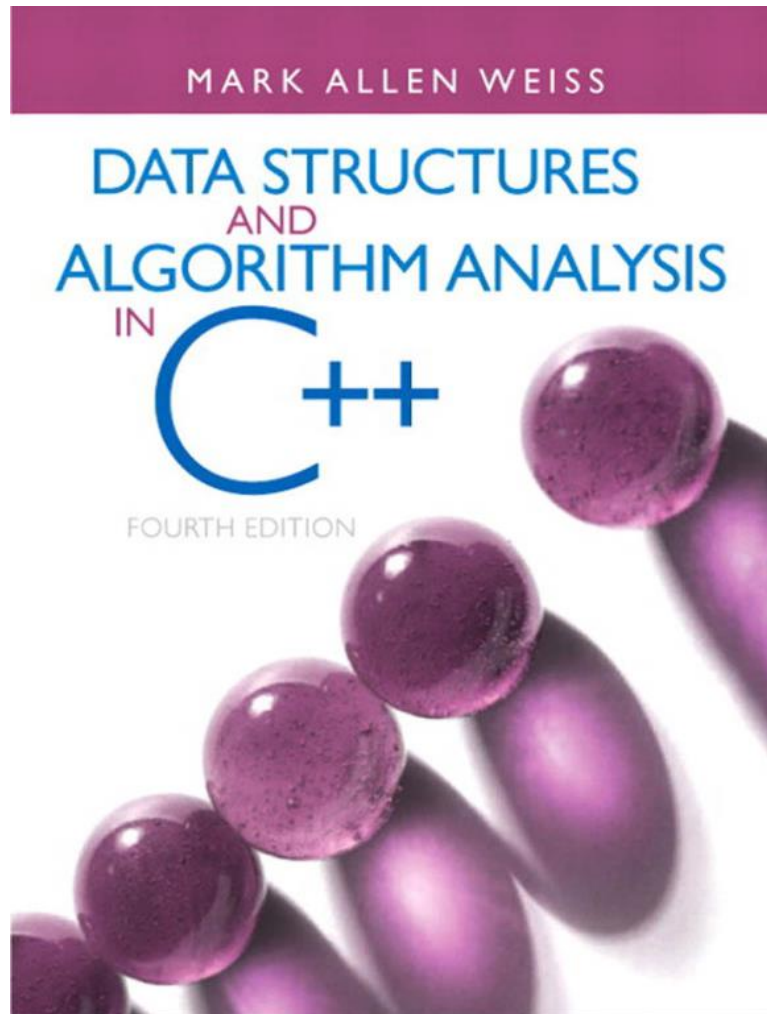


Assignment 3 Inclass : From book “Data Structures and Algorithms Analysis in C++”

by Mark Allen Weiss, p.283



Exercises

- 6.1 Can both insert and findMin be implemented in constant time?
- 6.2  a. Show the result of inserting 10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7, 4, 11, 13, and 2, one at a time, into an initially empty binary heap.
b. Show the result of using the linear-time algorithm to build a binary heap using the same input.
-  6.3 Show the result of performing three deleteMin operations in the heap of the previous exercise.

[คำอธิบายโจทย์และแนวทางการแก้ปัญหา **Binary Heap** (Exercises 6.2a & 6.3)]:

* หัวข้อแบบฝึกหัด (Inclass Exercises): Mark Allen Weiss, p.283

* Ex 6.2a: สร้าง Min-Binary Heap จากการ Insert ข้อมูล 15 ตัวตามลำดับ:

[10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7, 4, 11, 13, 2]

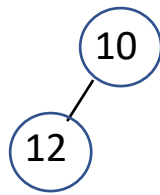
* Ex 6.3 : ทำการลบค่าต่ำสุด (deleteMin) ออกจาก Heap จำนวน 3 ครั้งติดต่อกัน

* กฎของ Min-Heap: โหนดลูกทุกโหนดต้องมีค่ามากกว่าหรือเท่ากับโหนดแม่เสมอ ($A[i] \leq A[2i], A[2i+1]$)

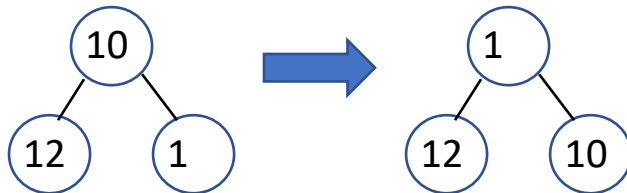
* โครงสร้างข้อมูล Array: เก็บข้อมูลเริ่มจาก Index 1 เพื่อให้ Parent = $i//2$ และ Children = $2i, 2i+1$

Solution:

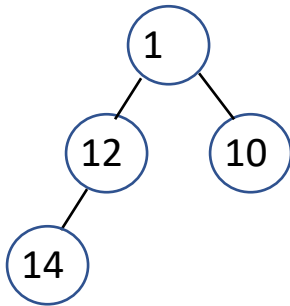
First step: From an empty binary heap, insert 10 then 12 :Check structure prop. and order prop.



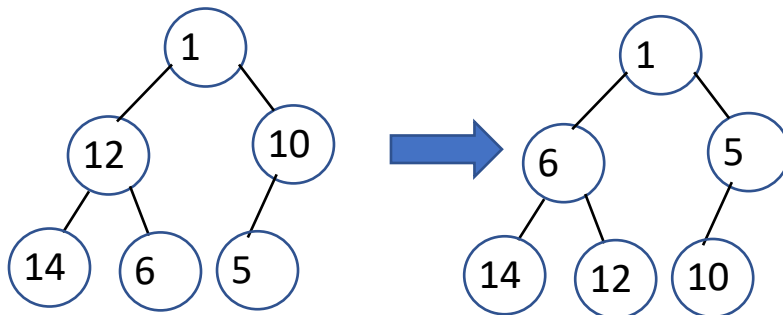
2nd step: Insert 1: Check structure prop. and order prop. Swap 10 1



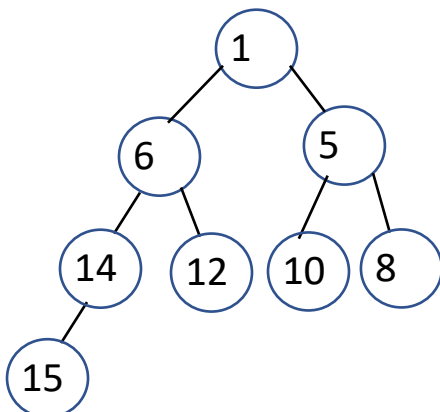
3rd step: Insert 14: Check structure prop. and order prop. No change.



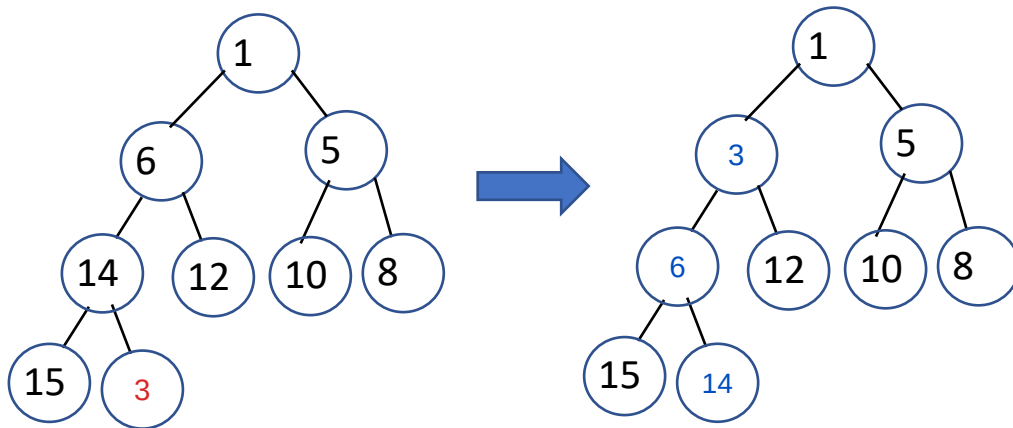
4th step: Insert 6 then 5: Check structure prop. and order prop. Swap 12 6, 10 5.



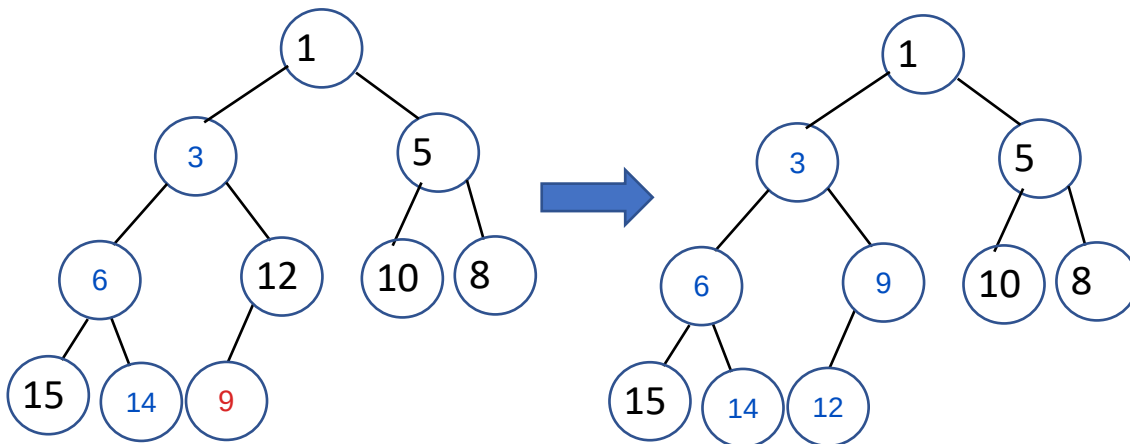
5th step: Insert 8 then 15: Check structure prop. and order prop. No change.



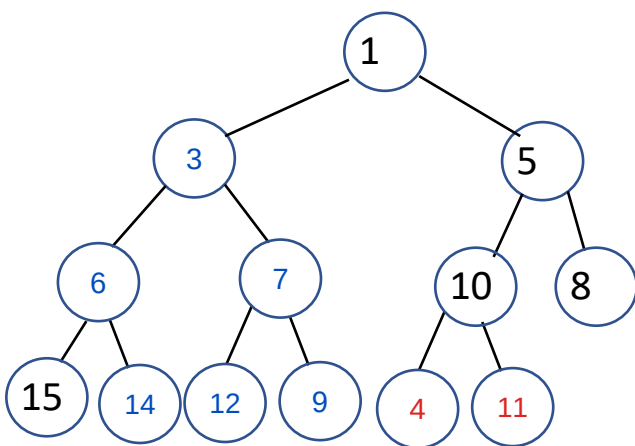
6th step: Insert 3: Check structure prop. and order prop.



7th step: Insert 9: Check structure prop. and order prop.



8th step: Insert 7 then 4 then 11: Check structure prop. and order prop.



9th step: Insert 13 then 2: Check structure prop. and order prop.

[ขั้นตอนละเอียดสำหรับ Step 9 (Insert 13 & 2) และการ Percolate Up 2 ครั้ง]:

* 9th step: Insert 13 then 2 (การแทรก 2 ตัวสุดท้าย):

1. Insert 13: ใส่ที่ Index 14 (โหนดลูกซ้ายของ 4) -> $13 > 4$ (Order ถูกต้อง ไม่ต้องสลับ)

2. Insert 2 : ใส่ที่ Index 15 (โหนดลูกขวาของ 4) -> เกิดการ Percolate Up ดังนี้:

* $2 < 8$ (Parent ที่ idx 7 คือ 8) -> สลับ 8 กับ 2 (2 ขยับขึ้นไปที่ Index 7)

* $2 < 4$ (Parent ที่ idx 3 คือ 4) -> สลับ 4 กับ 2 (2 ขยับขึ้นไปที่ Index 3)

* $2 > 1$ (Parent ที่ idx 1 คือ 1) -> หยุด! ได้โครงสร้าง Min-Heap ที่สมบูรณ์แบบ

-> ลำดับ Array หลัง Insert ครบ 15 ตัว: [1, 3, 2, 6, 7, 5, 4, 15, 14, 12, 9, 10, 11, 13, 8]

Finally: array of binary heap after insert all elements already.

Then 6.3

[0]	-	[0]	-
1	1	1	4
2	3	2	6
3	2	3	5
4	6	4	13
5	7	5	7
6	5	6	10
7	4	7	8
8	15	8	15
9	14	9	14
10	12	10	12
11	9	11	9
12	10	12	11
13	11	13	-
14	13	14	-
15	8	15	-
16	-	16	-
17	-	17	-

[การวิเคราะห์ Percolate Down ของ 3 deleteMin]:

* ขั้นตอนการทำงานของ 3 deleteMin Operations (Ex 6.3):

1. deleteMin #1 (ลบค่าต่ำสุด 1):

- * นำสมาชิกตัวสุดท้ายคือ 8 มาวางแทนที่ Root (Index 1)
- * **Percolate Down:** เปรียบเทียบลูกซ้าย (3) กับลูกขวา (2) -> เลือกตัวเล็กสุดคือ 2
- * $8 > 2 \rightarrow$ สลับ 8 กับ 2; ต่อมา 8 เปรียบเทียบกับลูก (5 และ 4) -> สลับ 8 กับ 4
- * ผลลัพธ์: [2, 3, 4, 6, 7, 5, 8, 15, 14, 12, 9, 10, 11, 13] (ขนาด 14)

2. deleteMin #2 (ลบค่าต่ำสุด 2):

- * นำสมาชิกตัวสุดท้ายคือ 13 มาวางแทนที่ Root (Index 1)
- * **Percolate Down:** เลือกลูกตัวเล็กสุดระหว่าง 3 กับ 4 -> เลือก 3
- * $13 > 3 \rightarrow$ สลับ 13 กับ 3; ต่อมาเลือกลูกของ 13 (6 และ 7) -> สลับ 13 กับ 6
- * ผลลัพธ์: [3, 6, 4, 13, 7, 5, 8, 15, 14, 12, 9, 10, 11] (ขนาด 13)

3. deleteMin #3 (ลบค่าต่ำสุด 3):

- * นำสมาชิกตัวสุดท้ายคือ 11 มาวางแทนที่ Root (Index 1)
- * **Percolate Down:** เลือกลูกตัวเล็กสุดระหว่าง 6 กับ 4 -> เลือก 4
- * $11 > 4 \rightarrow$ สลับ 11 กับ 4; ต่อมาเลือกลูกของ 11 (5 และ 8) -> สลับ 11 กับ 5
- * 11 เปรียบเทียบกับลูกคือ 10 -> สลับ 11 กับ 10
- * ผลลัพธ์สุดท้าย: [4, 6, 5, 13, 7, 10, 8, 15, 14, 12, 9, 11] (ขนาด 12)

* สรุปค่า Min ที่ถูกลบออกตามลำดับ: 1, 2, 3

* คุณสมบัติ Heap Order: Root มีค่าน้อยที่สุดเสมอ ($O(1)$ findMin, $O(\log N)$ deleteMin)